

IT342 – Design Patterns: Refactoring Report

PayWeac | Carin | Branch: feature/design-patterns-refactor

GitHub Repository: <https://github.com/fredjohnc909-blip/IT342-Carin-Payweac>

Branch: feature/design-patterns-refactor

Patterns Applied: Singleton, Factory Method, Adapter, Facade, Observer, Strategy

Refactoring 1 — Singleton Pattern (ApiClient.kt)

Pattern	Singleton
Location	mobile/app/src/main/java/com/carin/payweac/api/ApiClient.kt
Commit Msg	Applied Singleton Pattern to ApiClient (Retrofit instance)

Before — The Problem

The original code risked creating a **new Retrofit instance every time** an activity needed to make an API call. This wastes memory, creates multiple connection pools, and may lead to inconsistent configuration across screens.

```
// BEFORE – new instance created anywhere it was needed
val retrofit = Retrofit.Builder()
    .baseUrl("http://10.0.2.2:8080/")
    .addConverterFactory(GsonConverterFactory.create())
    .build()
val service = retrofit.create(ApiService::class.java)
// ■ Repeated in RegisterActivity AND LoginActivity – two instances!
```

After — Singleton Applied

Using Kotlin's **object** declaration, the Retrofit instance is created once and shared across the entire app. The **by lazy** delegate ensures thread-safe, deferred initialisation.

```
// AFTER – object keyword = JVM-level Singleton
object ApiClient {
    private const val BASE_URL = "http://YOUR_BACKEND_URL/"
    val instance: ApiService by lazy {
        Retrofit.Builder()
            .baseUrl(BASE_URL)
            .addConverterFactory(GsonConverterFactory.create())
            .build().create(ApiService::class.java)
    }
}
// ■ All Activities use: ApiClient.instance.login(...)
```

Improvement

- Memory usage reduced — only one Retrofit instance exists in the JVM.
- Configuration is centralised — change `BASE_URL` in one place.
- Thread-safe via Kotlin's `lazy{} delegate`.

Refactoring 2 — Factory Method (NotificationFactory)

Pattern	Factory Method
Location	src/main/kotlin/.../service/notification/
Commit Msg	Applied Factory Method Pattern to Notification Service

Before — The Problem

The original notification logic lived inside a single service method with a long **if/else chain**. Adding a new channel (SMS, in-app) required modifying the existing method, violating the Open/Closed Principle.

```
// BEFORE – monolithic if/else in NotificationService
fun sendNotification(channel: String, to: String, msg: String) {
    if (channel == "email") {
        // send email directly here – 20 lines
    } else if (channel == "push") {
        // send push directly here – 15 lines
    }
    // ■ Must edit this method for every new channel
}
```

After — Factory Method Applied

```
// AFTER – interface + factory
interface NotificationSender {
    fun send(to: String, message: String)
}

@Component class EmailSender : NotificationSender {
    override fun send(to: String, message: String) {
        // JavaMail logic here
    }
}

@Component class PushSender : NotificationSender {
    override fun send(to: String, message: String) {
        // FCM logic here
    }
}

object NotificationFactory {
    fun create(channel: String): NotificationSender = when (channel) {
        "email" -> EmailSender()
        "push" -> PushSender()
        else -> throw IllegalArgumentException("Unknown channel: $channel")
    }
}
```

```
// ■ New channel = new class only, no changes to existing code
```

Improvement

- Open/Closed Principle respected — new channels added without modifying existing code.
- Each sender class is independently testable.
- Service layer is decoupled from notification delivery details.

Refactoring 3 — Adapter Pattern (Payment Gateway)

Pattern	Adapter
Location	src/main/kotlin/.../payment/PayMongoAdapter.kt
Commit Msg	Applied Adapter Pattern to Payment Gateway Integration

Before — The Problem

The `PaymentService` called the third-party `PayMongo` SDK directly throughout the code. The SDK's raw method signatures (integers in centavos, snake_case fields) polluted the business logic, and swapping to another gateway would require changes everywhere.

```
// BEFORE — PaymentService tightly coupled to PayMongo SDK
fun processPayment(amount: Double) {
    val sdk = PayMongoSDK(apiKey)
    val centavos = (amount * 100).toInt()
    val pi = sdk.create_payment_intent(centavos, "PHP", "paymongo_desc")
    // ■ SDK types and naming leaking into business layer
```

After — Adapter Applied

```
// AFTER — clean interface + adapter
interface PaymentGateway {
    fun charge(amount: Double, currency: String): String
}

class PayMongoAdapter(private val sdk: PayMongoSDK) : PaymentGateway {
    override fun charge(amount: Double, currency: String): String {
        val centavos = (amount * 100).toInt()
        return sdk.create_payment_intent(centavos, currency, "PayWeac payment")
    }
}

@Service
class PaymentService(private val gateway: PaymentGateway) {
    fun processPayment(amount: Double): String {
        return gateway.charge(amount, "PHP")
    }
    // ■ No SDK details in service — swap gateway by injecting a new adapter
}
```

Improvement

- Business logic is isolated from third-party SDK changes.
- Switching payment providers requires only a new Adapter class.
- `PaymentGateway` interface is easy to mock in unit tests.

Refactoring 4 — Facade Pattern (AuthFacade)

Pattern	Facade
Location	src/main/kotlin/.../auth/AuthFacade.kt
Commit Msg	Applied Facade Pattern to Auth Controller — introduced AuthFacade

Before — The Problem

The AuthController directly called UserRepository, BCryptPasswordEncoder, and JwtUtil, making the controller bloated and business logic hard to reuse elsewhere (e.g., in an admin endpoint).

```
// BEFORE – fat controller
@PostMapping("/register")
fun register(@RequestBody req: RegisterRequest): ResponseEntity<*> {
    val existing = userRepository.findByEmail(req.email)
    if (existing != null) return ResponseEntity.badRequest().body("Exists")
    val hashed = BCrypt.hashpw(req.password, BCrypt.gensalt())
    val user = User(name=req.name, email=req.email, password=hashed)
    userRepository.save(user)
    // ■ 20+ lines of logic directly in controller
}
```

After — Facade Applied

```
// AFTER – thin controller
@RestController @RequestMapping("/api/auth")
class AuthController(private val authFacade: AuthFacade) {
    @PostMapping("/register")
    fun register(@RequestBody req: RegisterRequest) =
        ResponseEntity.ok(authFacade.register(req))

    @PostMapping("/login")
    fun login(@RequestBody req: LoginRequest) =
        ResponseEntity.ok(authFacade.login(req))
}

// AuthFacade handles all orchestration internally
@Service
class AuthFacade(private val userService: UserService,
    private val encoder: PasswordEncoder,
    private val jwtService: JwtService) {
    fun register(req: RegisterRequest): RegisterResponse { ... }
    fun login(req: LoginRequest): LoginResponse { ... }
}
```

Improvement

- Controller reduced from 60+ lines to under 10 lines.
- AuthFacade is reusable from any controller or scheduled task.
- Separation of concerns — controller handles HTTP, facade handles logic.

Refactoring 5 — Observer Pattern (Transaction Events)

Pattern	Observer (Spring ApplicationEvent)
Location	src/main/kotlin/.../transaction/
Commit Msg	Applied Observer Pattern to Transaction Service — event-driven notifications

Before — The Problem

The TransactionService directly called EmailService and AuditService after saving a transaction. This created tight coupling — adding a new reaction (e.g., push notification) required modifying the core transaction logic.

```
// BEFORE — tightly coupled side-effects
fun processPayment(userId: Long, amount: Double) {
    transactionRepository.save(Transaction(userId, amount))
    emailService.sendConfirmation(userId, amount) // ■ direct dependency
    auditService.logTransaction(userId, amount) // ■ direct dependency
}
```

After — Observer Applied

```
// AFTER — event-driven with Spring ApplicationEvent
data class TransactionCompletedEvent(
    val source: Any, val userId: Long, val amount: Double
) : ApplicationEvent(source)

@Service
class TransactionService(private val publisher: ApplicationEventPublisher,
    private val repo: TransactionRepository) {
    fun processPayment(userId: Long, amount: Double) {
        repo.save(Transaction(userId, amount))
        publisher.publishEvent(TransactionCompletedEvent(this, userId, amount))
        // ■ No knowledge of who listens
    }
}

@Component class EmailNotificationListener {
    @EventListener
    fun handle(e: TransactionCompletedEvent) { /* send email */ }
}

@Component class AuditLogListener {
    @EventListener
    fun handle(e: TransactionCompletedEvent) { /* write audit log */ }
}
```

Improvement

- TransactionService has zero knowledge of notification or audit logic.
- New reactions (SMS, push, analytics) are added as new @Component listeners — no core changes.

- Listeners can be individually unit-tested.

Refactoring 6 — Strategy Pattern (Fee Calculation)

Pattern	Strategy
Location	src/main/kotlin/.../payment/fee/
Commit Msg	Applied Strategy Pattern to Payment Fee Calculation

Before — The Problem

Fee calculation logic used a growing chain of if/else conditions inside `PaymentService`. Every new transaction type required editing the same method, increasing the risk of regression bugs.

```
// BEFORE – if/else fee logic in PaymentService
fun calculateFee(amount: Double, type: String): Double {
    return if (type == "basic") {
        15.0
    } else if (type == "premium") {
        amount * 0.015
    } else if (type == "international") {
        amount * 0.03 + 50.0
    } else { 0.0 } // ■ grows unbounded with each new type
```

After — Strategy Applied

```
// AFTER – Strategy Pattern
interface FeeStrategy {
    fun calculate(amount: Double): Double
}

class FlatFeeStrategy(private val fee: Double) : FeeStrategy {
    override fun calculate(amount: Double) = fee
}

class PercentageFeeStrategy(private val rate: Double) : FeeStrategy {
    override fun calculate(amount: Double) = amount * rate
}

class InternationalFeeStrategy : FeeStrategy {
    override fun calculate(amount: Double) = amount * 0.03 + 50.0
}

@Service
class PaymentService {
    private val strategies = mapOf(
        "basic" to FlatFeeStrategy(15.0),
        "premium" to PercentageFeeStrategy(0.015),
        "international" to InternationalFeeStrategy()
    )
    fun processPayment(amount: Double, type: String): Double {
        val fee = strategies[type]?.calculate(amount) ?: 0.0
        return amount + fee
    }
    // ■ New type = new class, zero changes to PaymentService
```

```
}  
}
```

Improvement

- Each fee rule is isolated in its own class — independently testable.
- PaymentService requires zero modification to add a new fee type.
- Fee strategies can be configured via Spring beans for runtime flexibility.

Overall Refactoring Summary

Pattern	File / Location	Problem Solved	Key Gain
Singleton	ApiClient.kt	Multiple Retrofit instances	1 shared instance, central config
Factory Method	NotificationFactory.kt	if/else notification sprawl	OCP — new channel = new class
Adapter	PayMongoAdapter.kt	SDK leaking into business layer	Swap gateway, easy mocking
Facade	AuthFacade.kt	Fat controller, repeated logic	Thin controller, reusable service
Observer	TransactionService.kt	Tight coupling to side-effects	Decoupled, extensible events
Strategy	FeeStrategy.kt	Growing if/else fee logic	Isolated, testable fee rules

Branch: feature/design-patterns-refactor | Commit message format used: "Applied [Pattern] Pattern to [Component]"